Adaptive Memory Research Assistant: A Hybrid AI Assistant Integrating Semantic Memory with PDF-Based Retrieval-Augmented Generation

Tanjila Ahmed Medha
Department of linguistics
tanjila.medha05@gmail.com

Abstract

Adaptive Memory Research Assistant that combines conversational memory, semantic retrieval, and PDF-based document understanding within a unified AI agent. The system was developed to improve contextual reasoning using Information Retrieval (IR) techniques such as vector embeddings, semantic search, and Retrieval-Augmented Generation (RAG). The assistant supports both general conversational interaction and research-oriented question answering from uploaded PDF documents. The system uses Sentence Transformers for embedding generation and ChromaDB as a vector database for storing conversational memories and document chunks. Uploaded PDFs are processed, divided into semantic chunks, and retrieved dynamically during question answering. In addition, previous conversations are embedded and stored to enable adaptive retrieval of conversational memory. The retrieved context is combined with the user query and passed to the `meta-llama/Llama-3.1-8B-Instruct` model via the Berget AI API. The project demonstrates how IR-based retrieval pipelines can improve contextual awareness in AI agents. The development process also highlighted practical IR challenges including retrieval contamination, chunk-size optimization, and context management.

Code and Demo

Github:medhuuuu/adaptive-memory-research-assistant-assignment2

Demo Video: `https://youtu.be/7cbRBa0Zrgs`
App link: adaptive-memory-research-assistant

1 Introduction

Large Language Models (LLMs) excel at natural language generation but suffer from a well-known limitation: they lack persistent memory across conversations and cannot dynamically access user-specific or document-level knowledge at inference time [1]. Retrieval-Augmented Generation [4] addresses this by coupling generation with an external retrieval step, enabling the model to ground its responses in relevant retrieved passages.

Existing systems typically implement RAG over static document corpora *or* maintain short-term session context, but rarely both simultaneously. This work proposes a unified architecture that: (1) persists and retrieves prior conversational exchanges as semantic memories, and (2) supports user-uploaded PDF documents as a queryable knowledge base—with both retrieval streams fused into a single prompt context at inference time.

The system targets users who need an assistant that *remembers* their interests and preferences across sessions while also being able to answer specific questions grounded in uploaded research documents.

2 System Architecture

The system follows a modular pipeline design with five primary components, implemented in Python with a Streamlit front end.

2.1 Conversational Memory Module

Every user–assistant exchange is stored as a text record. Before storage, the text is encoded into a dense vector representation using the `all-MiniLM-L6-v2` sentence transformer [6]—a compact but effective model for semantic similarity tasks. These embeddings are persisted in a `ChromaDB` collection.

At query time, the user's new input is embedded and compared against the stored memory collection via cosine similarity search. The top-k retrieved memories are prepended to the prompt as conversational context, allowing the LLM to recall relevant prior interactions:

$$\text{sim}(\mathbf{q}, \mathbf{m}_i) = \frac{\mathbf{q} \cdot \mathbf{m}_i}{\|\mathbf{q}\| \, \|\mathbf{m}_i\|} \tag{1}$$

where $\mathbf{q}$ is the query embedding and $\mathbf{m}_i$ is the i-th stored memory embedding.

2.2 PDF Research Assistant (RAG Pipeline)

Users can upload PDF documents through the Streamlit sidebar. Upon upload, text is extracted using PyPDF [5], then split into overlapping chunks of 1,500 characters. Each chunk is embedded and stored in a separate ChromaDB collection. When the user poses a question, relevant chunks are retrieved semantically and injected into the LLM prompt, forming a standard RAG pipeline [4].

A critical design decision was to *clear* the PDF collection on each new upload to prevent retrieval contamination—a scenario where chunks from a previous document pollute answers about a newly uploaded one.

2.3 Hybrid Retrieval and Prompt Construction

The novel contribution of this system is the simultaneous use of both retrieval sources. Given a query q, the system retrieves:

- C_{mem}: top-k semantically relevant memory records
- C_{pdf}: top-k relevant PDF chunks

Both are concatenated into a structured prompt template alongside the live user query. This enables cross-source reasoning, such as: *"Compare this uploaded paper with my interest in NLP"*—a query that requires knowledge from both retrieved memory and document content.

2.4 LLM Backend

The system uses `Berget AI` as the LLM provider via an OpenAI-compatible API endpoint. The underlying model is `meta-llama/Llama-3.1-8B-Instruct` [7], an 8-billion-parameter instruction-tuned variant of Llama 3.1. This model was chosen for its strong instruction-following capability, suitability for conversational and RAG workloads, and efficiency within the free credit tier provided for the assignment. The model handles four tasks: conversational response generation, memory-aware dialogue, PDF question answering, and contextual summarisation.

2.5 Prompting Strategy

Rather than passing the raw user query directly to the LLM, the system constructs a composite prompt at each turn. Retrieved conversational memories, retrieved PDF chunks, and the current user query are concatenated into clearly delimited sections before the API call is made. This structured prompt design is essential for the model to distinguish between background memory, document evidence, and the live instruction—reducing hallucination and improving answer fidelity [2].

3 Implementation

3.1 Technology Stack

Component	Technology
Frontend	Streamlit
LLM API	Berget AI (OpenAI-compatible)
Embedding Model	`all-MiniLM-L6-v2` [6]
Vector Database	ChromaDB
PDF Extraction	PyPDF
Language	Python 3

3.2 Module Structure

The codebase follows a clean separation of concerns:

- `app.py` — Streamlit UI and session state management
- `src/chatbot.py` — LLM API calls and response handling
- `src/embeddings.py` — Embedding generation via Sentence Transformers
- `src/memory_manager.py` — Memory storage and retrieval logic
- `src/pdf_handler.py` — PDF parsing, chunking, and indexing
- `src/retriever.py` — Unified hybrid retrieval interface

4 Challenges and Solutions

4.1 Retrieval Contamination

When a second PDF was uploaded without clearing the vector store, the assistant incorrectly blended content from both documents. The solution was to implement an explicit collection-clear function in `pdf_handler.py` that deletes all stored vectors before indexing a new document.

4.2 Chunk Size Tuning

Initial experiments with 500-character chunks produced low retrieval quality: chunks were too small to carry sufficient semantic context, leading to incomplete and disconnected answers for academic documents. Increasing chunk size to 1,500 characters substantially improved retrieval coherence, consistent with findings in the RAG literature [2].

4.3 Context Noise from Hybrid Retrieval

Fusing memory context, PDF chunks, and the live query into a single prompt risked producing an overly long or incoherent context window. This was mitigated through prompt engineering: clearly delimited sections for each context type, and a fixed token budget per retrieval source to prevent either stream from dominating.

5 Discussion

The hybrid retrieval design differentiates this system from standard PDF chatbots or pure conversational agents. Most open-source RAG implementations treat retrieval as a single-source problem. By maintaining two independent vector collections and merging their outputs at the prompt level, the system supports richer, cross-referential queries.

The use of `all-MiniLM-L6-v2` offers a practical trade-off: it is fast enough for real-time retrieval in a Streamlit application while producing embeddings of sufficient quality for academic text [6]. Future work could explore larger embedding models or sparse-dense hybrid retrieval such as ColBERT [3] to further improve precision.

6 Conclusion

This paper presented the Adaptive Memory Research Assistant, a hybrid AI system integrating conversational memory retrieval and PDF-based RAG into a unified interface. The system demonstrates that combining multiple retrieval streams with careful prompt engineering enables context-aware AI assistance beyond what either stream provides alone. Lessons learned—particularly around retrieval contamination and chunk size tuning—offer practical guidance for practitioners building similar systems.

References

[1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

[2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024.

[3] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, Virtual Event, China, 2020. ACM.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[5] py-pdf contributors. PyPDF: A pure-python PDF library. `https://github.com/py-pdf/pypdf`, 2023. Accessed: 2024.

[6] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China, 2019. Association for Computational Linguistics.

[7] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.